第 11 节 实操任务 2：实现 Reviser + Hum...

> **学习目标**：review 节点有 3 路分支（organize / revise / human_flag），revise → review 循环能闭合
> 目标文件：`workflows/reviser.py` + `workflows/human_flag.py` + 修改 `workflows/state.py` + 修改 `workflows/graph.py`

背景

实操 1 把 Reviewer 升级成了 5 维加权评分。但 Reviewer 只会打分，**不会改内容**—— 需要 **Reviser Agent** 读反馈去定向修改 analyses。

同时循环必须有出口——不能无限重试——需要 **HumanFlag Agent** 兜底。
本实操做三件事：

1. 创建 `workflows/reviser.py`（只修改不评估）
2. 创建 `workflows/human_flag.py`（兜底退出）
3. 修改 `state.py` + `graph.py` 把 revise 循环 + human_flag 分支接进来

步骤 1：创建 reviser.py

提示词：

```
请帮我编写 workflows/reviser.py 中的 revise_node 函数：

需求：
1. 读 state["analyses"] 和 state["review_feedback"]
2. 把 feedback 注入修改 prompt
3. 调 LLM 返回修改后的 analyses 列表
4. temperature=0.4（允许创造性改写）
5. analyses 或 feedback 空时跳过（返回 {}）
6. 返回 {"analyses": improved, "cost_tracker": tracker}
```

参考实现： `workflows/reviser.py`

```
"""Reviser Agent — 定向修改节点（只修改不评估）

Reviser 和 Reviewer 是两个独立 Agent —— 避免 Reviewer 给自己打高分。
"""

import json

from workflows.model_client import accumulate_usage, chat_json
```

```
from workflows.state import KBState


def revise_node(state: KBState) -> dict:
    """Reviser 节点：根据 Reviewer 反馈定向修改 analyses"""
    analyses = state.get("analyses", [])
    feedback = state.get("review_feedback", "")
    iteration = state.get("iteration", 0)
    tracker = state.get("cost_tracker", {})

    if not analyses or not feedback:
        print("[Reviser] 无可修改内容，跳过")
        return {}

    prompt = f"""你是知识库编辑。以下是审核员的反馈，请据此修改这些分析结果。

【审核反馈】
{feedback}

【当前分析结果】
{json.dumps(analyses, ensure_ascii=False, indent=2)}

【修改要求】
- 重点改进反馈中提到的弱项维度
- 保留已经不错的部分
- 保持相同字段结构
- 返回修改后的 JSON 数组"""

    try:
        improved, usage = chat_json(
            prompt,
            system="你是经验丰富的知识库编辑。根据反馈定向修改，不要过度发散。",
            temperature=0.4,
        )
        tracker = accumulate_usage(tracker, usage)
        if isinstance(improved, list) and improved:
            print(f"[Reviser] 定向修改 {len(improved)} 条 analyses (迭代 {
            return {"analyses": improved, "cost_tracker": tracker}
    except Exception as e:
        print(f"[Reviser] 修改失败: {e}")

    return {"cost_tracker": tracker}
```

步骤 2：创建 human_flag.py

循环必须有出口。超过 `max_iterations` 还没通过，说明问题不在“质量”而在“数据”—— 需要人工判断。HumanFlag 节点把问题条目写到独立目录，不污染主知识库。

参考实现： `workflows/human_flag.py`

```
"""HumanFlag Agent — 人工介入节点（异常终点）"""

import json
import os
from datetime import datetime, timezone

from workflows.state import KBState


def human_flag_node(state: KBState) -> dict:
    """审核循环超过上限时的兜底 —— 写入 pending_review/ 目录"""
    analyses = state.get("analyses", [])
    iteration = state.get("iteration", 0)
    feedback = state.get("review_feedback", "")

    print(f"[HumanFlag] ⚠️ 达到 {iteration} 次审核仍未通过")
    print(f"[HumanFlag] 最后反馈: {feedback[:200]}")

    base = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    pending_dir = os.path.join(base, "knowledge", "pending_review")
    os.makedirs(pending_dir, exist_ok=True)

    today = datetime.now(timezone.utc).strftime("%Y-%m-%d-%H%M%S")
    filepath = os.path.join(pending_dir, f"pending-{today}.json")
    with open(filepath, "w", encoding="utf-8") as f:
        json.dump({
            "timestamp": today,
            "iterations_used": iteration,
            "last_feedback": feedback,
            "analyses": analyses,
        }, f, ensure_ascii=False, indent=2)

    print(f"[HumanFlag] 已保存到 {filepath}")
    return {"needs_human_review": True}
```

步骤 3：更新 state.py 添加 needs_human_review 字段

在 `workflows/state.py` 的 KBState 里加一行：

```
class KBState(TypedDict):
    plan: dict
    sources: list[dict]
    analyses: list[dict]
```

```
    articles: list[dict]
    review_feedback: str
    review_passed: bool
    iteration: int
    needs_human_review: bool   # ← 新增：HumanFlag 节点设为 True
    cost_tracker: dict
```

步骤 4：更新 graph.py 为 3 路条件路由

提示词：

```
请修改 workflows/graph.py 支持 3 路条件路由：
1. import revise_node 和 human_flag_node
2. 注册为节点 "revise" 和 "human_flag"
3. 重写路由函数 should_continue → route_after_review，返回 3 个分支：
   - 通过 → "organize"
   - 不通过且 iteration < 3 → "revise"
   - 不通过且 iteration >= 3 → "human_flag"
4. 添加 graph.add_edge("revise", "review") 形成循环
5. 添加 graph.add_edge("human_flag", END)
```

参考实现： `workflows/graph.py`

```python
"""LangGraph 工作流图 — 第 11 节 6 节点版"""

from langgraph.graph import END, StateGraph

from workflows.analyzer import analyze_node
from workflows.collector import collect_node
from workflows.human_flag import human_flag_node
from workflows.organizer import organize_node
from workflows.reviewer import review_node
from workflows.reviser import revise_node
from workflows.state import KBState


def route_after_review(state: KBState) -> str:
    """条件路由：审核后 3 条出口"""
    if state.get("review_passed", False):
        return "organize"       # 通过 → 整理入库
    elif state.get("iteration", 0) >= 3:
        return "human_flag"     # 超限 → 人工介入
    else:
        return "revise"         # 未通过 → 定向修改


def build_graph() -> StateGraph:
```

```
    graph = StateGraph(KBState)

    graph.add_node("collect", collect_node)
    graph.add_node("analyze", analyze_node)
    graph.add_node("review", review_node)
    graph.add_node("revise", revise_node)
    graph.add_node("organize", organize_node)
    graph.add_node("human_flag", human_flag_node)

    graph.add_edge("collect", "analyze")
    graph.add_edge("analyze", "review")

    graph.add_conditional_edges(
        "review",
        route_after_review,
        {
            "organize": "organize",
            "revise": "revise",
            "human_flag": "human_flag",
        },
    )

    # revise 后回到 review 形成循环
    graph.add_edge("revise", "review")

    # 两个终点
    graph.add_edge("organize", END)
    graph.add_edge("human_flag", END)

    graph.set_entry_point("collect")
    return graph


app = build_graph().compile()
```

步骤 5：运行端到端测试

5.1 默认场景（正常通过）

```
cd ~/ai-knowledge-base/v3-multi-agent
python3 -m workflows.graph
```

期望输出：

```
[Collector] 采集到 10 条原始数据
[Analyzer] 完成 10 条分析
```

```
[Reviewer] 加权总分: 7.2/10, 通过: True (第 1 次审核)
[Organizer] 整理出 10 条知识条目(准备入库)
[Organizer] 已写入 10 篇到磁盘
```

5.2 强制触发 revise 分支

临时把 `workflows/reviewer.py` 里的 `REVIEWER_PASS_THRESHOLD` 改为 `9.0` ：

```
REVIEWER_PASS_THRESHOLD = 9.0  # 临时提高
```

再跑一次：

```
python3 -m workflows.graph
```

期望输出：

```
[Reviewer] 加权总分: 7.2/10, 通过: False (第 1 次审核)
[Reviser] 定向修改 5 条 analyses (迭代 1)
[Reviewer] 加权总分: 7.5/10, 通过: False (第 2 次审核)
[Reviser] 定向修改 5 条 analyses (迭代 2)
[Reviewer] 加权总分: 7.8/10, 通过: False (第 3 次审核)
[HumanFlag] ⚠️ 达到 3 次审核仍未通过
[HumanFlag] 已保存到 knowledge/pending_review/pending-xxx.json
```

测试完记得改回 `REVIEWER_PASS_THRESHOLD = 7.0` **！**

5.3 验证三条路径

路径	触发条件	期望终点
A：通过	默认阈值 7.0	`organize` → `knowledge/articles/`
B：循环后通过	阈值调到 7.5	`revise` → `review` → `organize`
C：HumanFlag	阈值调到 9.0	`human_flag` → `knowledge/pending_review/`

步骤 6：提交到 Git

```
git add workflows/reviser.py workflows/human_flag.py workflows/state.py
git commit -m "feat: add reviser + human_flag + 3-way conditional routin
```

步骤 7：验证清单

检查项	期望	实际
workflows/reviser.py 存在	是	
workflows/human_flag.py 存在	是	
KBState 有 needs_human_review 字段	是	
graph.py 的 route_after_review 有 3 个分支	是	
revise → review 边存在（形成循环）	是	
organize 和 human_flag 都连到 END	是	
三条路径都能触发	是	

完成！6 节点工作流 + 审核修正循环 + 人工兜底全部实现。下一个实操引入 **Planner Agent** 把 `max_iterations` 等参数从硬编码变成动态规划。